Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET B

Dataset B – Issue / Bug Tracking System

Assessment Objective

This assessment evaluates your ability to build a complete MERN Full Stack application involving:

- Authentication & Authorization
- External API Integration
- Data Validation & Sanitization
- MongoDB Persistence
- Relationship Modeling
- REST API Development
- Querying & Aggregation
- Frontend Integration
- Protected Routes
- State Management
- Deployment Readiness

The assessment focuses on engineering workflows rather than isolated CRUD operations.

Problem Statement

You are required to develop a:

Issue / Bug Tracking Management System

The system manages:

- Users
- Projects
- Issues / Bugs
- Assignments
- Comments / Activity Logs

Students must:

1. Fetch issue tracking dataset from external API
2. Validate and sanitize records
3. Persist valid records into MongoDB Atlas
4. Build secure REST APIs
5. Implement role-based authorization
6. Build React frontend consuming backend APIs
7. Implement analytics & dashboard features

Standard Response Structure

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": []
}
```

Error Response

```
{
  "success": false,
  "message": "Error message"
}
```

Section A — Authentication & Authorization

Objective

Implement JWT-based authentication and role-based access control.

Q1 — Register API

POST /auth/register

Requirements

- Register new user
- Password hashing mandatory
- Duplicate email prevention
- Store role

Roles

- admin
- manager
- developer
- tester

Q2 — Login API

POST /auth/login

Requirements

- JWT token generation
- Invalid credential handling
- Return authenticated user details

Q3 — Current User API

GET /auth/me

Requirements

- Protected route

- Return logged-in user details

Authorization Rules

Only:

- admin
- manager

can:

- create projects
- assign issues
- change issue priority
- access analytics APIs

Developers can:

- update assigned issues
- change issue status

Testers can:

- report bugs
- add comments

Section B — Dataset Synchronization & MongoDB Persistence

Objective

Fetch dataset from external API, validate records, sanitize values, and persist valid records into MongoDB Atlas.

Q4 — Sync API

POST /sync

Responsibilities

1. Fetch dataset from private API
2. Validate records
3. Normalize values
4. Reject invalid entries
5. Prevent duplicate insertion
6. Persist valid records into MongoDB
7. Return sync summary

Example Response

```
{
  "success": true,
  "totalFetched": 100,
  "inserted": 84,
  "duplicates": 6,
  "rejected": 10
}
```

Required Sanitization Examples

Raw Value	Expected Value
" critical "	"critical"
"OPEN"	"open"
" Login Bug "	"Login Bug"

Validation Requirements

Reject:

- invalid priority values
- invalid status values
- invalid dates
- invalid project references
- duplicate issue IDs
- invalid assigned users

Q5 — Health API

GET /health

Example

```
{
  "success": true,
  "database": "connected",
  "documentCount": 84
}
```

Section C — MongoDB Persistence, Relationships & Runtime Validation

Objective

Implement MongoDB persistence with proper relationships, validation logic, and runtime data consistency.

Required Collections & Relationships

User

Fields

- userId
- name
- email
- role
- department
- status

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET B

Project

Fields

- projectId
- title
- description
- owner
- members
- status
- startDate

Issue

Relationships

- project → Project
- assignedTo → User
- reportedBy → User

Fields

- issueId
- title
- description
- priority
- severity
- status
- dueDate

Comment

Relationships

- issue → Issue
- user → User

Fields

- commentId
- message
- createdAt

Activity Log

Relationships

- issue → Issue
- user → User

Fields

- action
- previousStatus
- newStatus
- timestamp

Section D — CRUD & Workflow APIs

Objective

Implement REST APIs with proper validation and workflow handling.

Q6 — User APIs

GET /users
GET /users/:id

Q7 — Project APIs

POST /projects
GET /projects
GET /projects/:id
PATCH /projects/:id
DELETE /projects/:id

Q8 — Issue APIs

POST /issues
GET /issues
GET /issues/:id
PATCH /issues/:id
DELETE /issues/:id

Q9 — Comment APIs

POST /comments
GET /comments
GET /comments/:id
DELETE /comments/:id

Workflow Rules

- Closed issues cannot be reassigned
- Duplicate issue titles within same project not allowed
- Only managers/admins can assign issues
- Developers can update only assigned issues
- Invalid project references must be rejected

Section E — Issue Workflow APIs

Objective

Implement protected workflow APIs.

Q10 — Assign Issue

PATCH /issues/:id/assign

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET B

Requirements

- Only admin/manager allowed
- Assigned user must exist
- Closed issues cannot be assigned

Q11 — Update Issue Status

PATCH /issues/:id/status

Allowed Status

- open
- in-progress
- testing
- resolved
- closed

Workflow Rules

- Closed issues cannot move back without reopen
- Resolved issues cannot be edited directly
- Testers cannot close issues directly
- Only assigned developer can move issue to testing

Section F — Filtering, Search & Pagination

Objective

Implement scalable querying APIs.

Q12 — Issue Filters

Examples

GET /issues?priority=high
GET /issues?status=open
GET /issues?severity=critical

Q13 — Project Filters

Examples

GET /projects?status=active
GET /projects?owner=Rahul

Q14 — Pagination & Search

Examples

GET /issues?page=1&limit=10
GET /issues?search=login

Combined Query Example

GET /issues?
status=open&
priority=high&
page=2&
limit=5

Section G — Analytics & Aggregation APIs

Objective

Implement MongoDB aggregation APIs.

Q15 — Issue Analytics

GET /analytics/issues

Return

- total issues
- open issues
- resolved issues
- closed issues

Q16 — Project Analytics

GET /analytics/projects

Return

- project-wise issue count
- active project count
- closed project count

Q17 — Developer Analytics

GET /analytics/developers

Return

- developer-wise resolved issues
- average resolution time
- highest resolved issue count

Section H — Frontend Integration

Objective

Build React frontend consuming backend APIs.

Mandatory Frontend Features

Authentication

- Login page
- JWT token handling
- Protected routes
- Role-based rendering

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET B

Dashboard

Display:

- issue analytics cards
- active projects
- recent issues
- pending tasks

Developer Module

- view assigned issues
- update issue status
- add comments
- filter issues

Manager Module

- manage projects
- assign issues
- update priorities
- access analytics

Mandatory State Management

Use:

- Context API
- Reducer

Section I — Frontend Evaluator Compliance, Global State & Protected Routes

Objective

Evaluate frontend architecture.

Requirements

Implement:

- Protected routes
- Role-based navigation
- Reducer logic
- Global state management
- Required data-testid attributes
- Dynamic UI updates based on global state

Mandatory Global State

Your application must maintain global state for:

- authenticated user
- JWT token
- users
- projects
- issues
- comments
- filters/search values
- analytics/dashboard data

Use:

- Context API
- Reducer

Mandatory data-testid Attributes

Authentication

Component	data-testid
Login form	login-form
Email input	email-input
Password input	password-input
Login button	login-btn
Logout button	logout-btn

Navigation

Component	data-testid
Navbar	navbar
Dashboard link	dashboard-link
Users link	users-link
Projects link	projects-link
Issues link	issues-link
Comments link	comments-link

Dashboard

Component	data-testid
Analytics container	analytics-container
Total issues card	total-issues-card
Active projects card	active-projects-card
Open issues card	open-issues-card
Closed issues card	closed-issues-card
Issue analytics chart	issue-chart
Recent activity section	recent-activity

Date: 03 / 06 / 2026

Duration: 3 hours

FINAL ASSESSMENT

III YEAR – SEMESTER 6

CSE23AE304 - PCP V

(B.Tech. E01,E02,E03,E04)

SET B

Issue Module

Component	data-testid
Issue table	issue-table
Issue search input	issue-search
Issue filter dropdown	issue-filter
Issue row	issue-row

Project Module

Component	data-testid
Project list	project-list
Create project button	create-project-btn
Assign issue button	assign-issue-btn
Project search input	project-search
Pagination next button	pagination-next
Pagination previous button	pagination-prev

Comment Module

Component	data-testid
Comment table	comment-table
Add comment button	add-comment-btn
Comment row	comment-row

Section J — Frontend Evaluator Compliance & Deployment

Objective

Validate deployment readiness, frontend evaluator compatibility, and runtime UI behavior using automated Playwright-based evaluation.

Mandatory Frontend Behavior

Your frontend must:

- consume your own backend APIs
- update UI dynamically after CRUD operations
- maintain JWT token in global state
- maintain authenticated session after refresh
- restrict unauthorized route access
- render UI based on user role

Mandatory React Router Routes

Students must implement the following routes using React Router DOM.

Route	Purpose
/login	Authentication
/dashboard	Analytics dashboard
/users	User management
/projects	Project management
/issues	Issue management
/comments	Comment management
/profile	Current user profile

Global State Exposure Requirement

To support automated Playwright evaluation, students must expose global application state using:

window.appState

The global state object must dynamically reflect the current application state.

Mandatory window.appState Structure

```
window.appState = {
  authUser,
  token,
  users,
  projects,
  issues,
  comments,
  filters,
  analytics
}
```